



Programming

Exceptions

Vassilis Markos, Mediterranean College

Week 08

Contents



- 1 Last Time's Exercises
- 2 Exceptions
- 3 Fun Time!

Last Time's Exercises

In-class Exercise #001



In today's materials, under the following link:

`../labs/Programming_Lab_03.pdf`

you can find the third part of a Lab series for this course. Follow the instructions found therein to start working on it!



Exceptions

What Might Go Wrong Here?



```
1 # source/whoops_001.py
2
3 if __name__ == "__main__":
4     a = float(input("Enter a: "))
5     b = float(input("Enter b: "))
6     c = a / b
7     print(f"{a} / {b} == {c}.")
```

What Might Go Wrong Here?

```
1 # source/whoops_001.py
2
3 if __name__ == "__main__":
4     a = float(input("Enter a: "))
5     b = float(input("Enter b: "))
6     c = a / b
7     print(f"{a} / {b} == {c}.")
```

- Division by zero, of course!

What Might Go Wrong Here?

```
1 # source/whoops_001.py
2
3 if __name__ == "__main__":
4     a = float(input("Enter a: "))
5     b = float(input("Enter b: "))
6     c = a / b
7     print(f"{a} / {b} == {c}.")
```

- Division by zero, of course!
- So, how to catch this?

Catching Errors



One way to handle errors as the above is to use a simple if or while block:

```
1 # source/whoops_002.py
2
3 if __name__ == "__main__":
4     a = float(input("Enter a: "))
5     b = float(input("Enter b: "))
6     while b == 0:
7         b = float(input("Invalid value, b == 0! Enter b: "))
8     c = a / b
9     print(f"{a} / {b} == {c}.")
```

Still, Something Might Go Wrong Here...



```
1 # source/whoops_001.py
2
3 if __name__ == "__main__":
4     a = float(input("Enter a: "))
5     b = float(input("Enter b: "))
6     c = a / b
7     print(f"{a} / {b} == {c}.")
```

Still, Something Might Go Wrong Here...

```
1 # source/whoops_001.py
2
3 if __name__ == "__main__":
4     a = float(input("Enter a: "))
5     b = float(input("Enter b: "))
6     c = a / b
7     print(f"{a} / {b} == {c}.")
```

- The user might enter values that cannot be parsed as floats!

Still, Something Might Go Wrong Here...

```
1 # source/whoops_001.py
2
3 if __name__ == "__main__":
4     a = float(input("Enter a: "))
5     b = float(input("Enter b: "))
6     c = a / b
7     print(f"{a} / {b} == {c}.")
```

- The user might enter values that cannot be parsed as floats!
- For instance, foo or one.

Still, Something Might Go Wrong Here...

```
1 # source/whoops_001.py
2
3 if __name__ == "__main__":
4     a = float(input("Enter a: "))
5     b = float(input("Enter b: "))
6     c = a / b
7     print(f"{a} / {b} == {c}.")
```

- The user might enter values that cannot be parsed as floats!
- For instance, foo or one.
- How to handle this?

Catching Exceptions



- There are several kinds of errors, but we can split them, in general, into two broad categories:
 - Compilation time errors.
 - Execution time errors.
- The first ones can be captured prior to program compilation (interpretation, if you prefer), so we might have ways to handle them in our code with a 100% success rate.
- For the latter, this might or might not be the case.
 - In the case of division by zero this is easily fixed by monitoring user input.
 - In the case of malformed input we need more sophisticated machinery...

Python Exceptions

One way to handle malformed user input is through Python exceptions:

```
1 # source/whoops_003.py
2
3 if __name__ == "__main__":
4     try:
5         a = float(input("Enter a: "))
6     except ValueError:
7         a = float(input("Please, enter a float for a: "))
8     b = float(input("Enter b: "))
9     while b == 0:
10         b = float(input("Invalid value, b == 0! Enter b: "))
11     c = a / b
12     print(f"{a} / {b} == {c}.")
```

Okay, But...



- ...a user might provide malformed input more than once.

Okay, But...



- ...a user might provide malformed input more than once.
- How to fix that?

Okay, But...

- ...a user might provide malformed input more than once.
- How to fix that?

```
1 # source/whoops_004.py
2 if __name__ == "__main__":
3     is_valid = False
4     while not is_valid:
5         try:
6             a = float(input("Enter a: "))
7             is_valid = True
8         except ValueError:
9             pass
10    b = float(input("Enter b: "))
11    while b == 0:
12        b = float(input("Invalid value, b == 0! Enter b: "))
13    c = a / b
14    print(f"{a} / {b} == {c}.")
```

pass



- So far we have used `pass` as a mysterious Python spell that stops the text editor from complaining when a function has been declared but not defined.
- However, `pass` is actually a directive that asks the compiler to just proceed with this piece of code and “pass” to the next directive.
- You can think of it as a “do nothing and go on” command.
- So, in the above case, we essentially keep track of value errors and do nothing in that case, other than asking again for user input — since we do not change the value of the `is_valid` flag.

A More Elegant Solution

```
1 # source/whoops_005.py
2
3 if __name__ == "__main__":
4     is_valid = False
5     msg_header = ""
6     while not is_valid:
7         try:
8             a = float(input(f"{msg_header}Enter a: "))
9             is_valid = True
10        except ValueError:
11            msg_header = "Malformed input. "
12    b = float(input("Enter b: "))
13    while b == 0:
14        b = float(input("Invalid value, b == 0! Enter b: "))
15    c = a / b
16    print(f"{a} / {b} == {c}.")
```

What About b?



- So far, we have properly handled input in the case of our first number, a.
- What about b?
- Of course we can just repeat the above piece of code but, what's the problem with that?

What About b?



- So far, we have properly handled input in the case of our first number, a.
- What about b?
- Of course we can just repeat the above piece of code but, what's the problem with that?
- **CODE REPETITION!**
- So, what would be a better solution?
- To use a function to make sure we properly read a float from users.

What About b?



- So far, we have properly handled input in the case of our first number, a.
- What about b?
- Of course we can just repeat the above piece of code but, what's the problem with that?
- **CODE REPETITION!**
- So, what would be a better solution?
- To use a function to make sure we properly read a float from users.
- Implement it!

One Way...



```
1 # source/whoops_006.py
2
3 def read_float(input_name):
4     is_valid = False
5     msg_header = ""
6     while not is_valid:
7         try:
8             x = float(input(f"{msg_header}Enter {input_name}: "))
9             is_valid = True
10        except ValueError:
11            msg_header = "Malformed input. "
12    return x
13
14 if __name__ == "__main__":
15     a = read_float("a")
16     b = read_float("b")
17     while b == 0:
18         b = read_float("b")
19     c = a / b
20     print(f"{a} / {b} == {c}.")
```

Catching Different Exceptions

What do you expect the following script to print?

```
1 # source/whoops_007.py
2
3 import random
4
5 if __name__ == "__main__":
6     fancy_numbers = { i: random.choice([True, False]) for i in range(100)}
7     try:
8         n = int(input("Enter an integer: "))
9         print(f"Is {n} fancy?: {fancy_numbers[n]}.")
10    except ValueError:
11        print(f"You messed up!")
12    except KeyError:
13        print(f"{n} is not eligible for fanciness.")
```

Catching Different Exceptions



- We can handle as many exceptions as we want to by using multiple `except` blocks.
- You can think of it as simple `if-elif` blocks.
- So, in case you know that two or more exceptions might be thrown from a certain piece of code, you can catch them and handle them separately in an entirely different way.
- **Tip:** Only handle exceptions you know they might occur and not just any exception you can think of!

A Brief Break?



Run and then terminate the following script using `Ctrl + C`:

```
1 # source/whoops_008.py
2
3 if __name__ == "__main__":
4     while True:
5         try:
6             s = input("Guess what I am thinking: ")
7             print("Wrong! Try again or quit (Ctrl + C).")
8         except KeyboardInterrupt:
9             print("\nNot that easy, right?")
10            pass
```

Or What?



- Since `except` blocks work like `if-elif` ones, what are we still missing?

Or What?

- Since `except` blocks work like `if-elif` ones, what are we still missing?
- `else` Blocks!
- With Python exceptions we can also use `else` to denote code that should run in case no exceptions occur:

```
1 # source/whoops_009.py
2
3 if __name__ == "__main__":
4     try:
5         x = float(input("Enter a float to compute its inverse: "))
6         inv = 1 / x
7     except ZeroDivisionError:
8         print("Zero has no inverse!")
9     else:
10        print(f"1 / {x} == {inv}")
```

Finally...



- What if we open a file and something goes wrong?

Finally...



- What if we open a file and something goes wrong?
- We should wrap things up and make sure we have closed the file!
- This means that we need a way to make sure that regardless of whether exceptions occur or not, we can perform a certain action before the program terminates its execution.
- Using Python exceptions, this is achieved by the `finally` block.

Raising An Exception



- In some cases, you might want to inform others (be it humans or other processes) that something has gone wrong.
- This can be achieved by raising an exception through the `raise` keyword.
- That is, we can interrupt, e.g., a function's execution by raising an exception that we let others to decide how to handle.
- This is quite useful when it comes to processes we know might be faulty but we also know how to fix.

What Will This Print?



```
1 # source/whoops_010.py
2
3 def foo():
4     try:
5         raise KeyboardInterrupt
6     finally:
7         print("Oh, noooo!")
8
9 if __name__ == "__main__":
10     foo()
```

A Few Final(ly) Notes



- If an exception occurs during execution of the try clause, the exception may be handled by an except clause. If the exception is not handled by an except clause, the exception is re-raised after the finally clause has been executed.
- An exception could occur during execution of an except or else clause. Again, the exception is re-raised after the finally clause has been executed.
- If the finally clause executes a break, continue or return statement, exceptions are not re-raised.

A Few Final(ly) Notes



- If the try statement reaches a break, continue or return statement, the finally clause will execute just prior to the break, continue or return statement's execution.
- If a finally clause includes a return statement, the returned value will be the one from the finally clause's return statement, not the value from the try clause's return statement.



Fun Time!

In-class Exercise #001



Revisit `source/whoops_006.py`:

- There we have created a function to safely read floats from users.
- Meanwhile, we have handled division by zero by repeatedly asking the user to provide valid input in case of zero denominators.
- Rewrite this part of code utilising Python exception handling.

In-class Exercise #002: Logging



One common practice regarding exceptions and exception handling is logging, i.e., the practice of keeping track of any exceptions into a file.

- Create a small Python script that asks user for some input and computes a simple output. It might be as simple as asking for two integers and computing their sum.
- Put it in a while loop and handle exceptions appropriately, making sure no error can interrupt its functionality other than `KeyboardInterrupt`.
- Keep logs of all errors. You can do this either manually or by using relevant python libraries, such as logging.

In-class Exercise #003



In today's materials, under the following link:

`../labs/Programming_Lab_04.pdf`

you can find the fourth part of a Lab series for this course. Follow the instructions found therein to start working on it!

In-class Exercise #004



Start working on all Labs found in today's materials `homework` directory.
To help me asses those files, you can name them as follows:

`task_xxx.py`

where `xxx` is the number of the task. For instance, task 5 file could be named `task_005.py`.

Submit your work via email at: `v.markos@mc-class.gr`

Homework



- In this week's materials, under the `homework` directory, you can find some Python programming Tasks. Complete as many of them as you can (preferably all).
- This is important, since tasks such as those provided with this lecture's materials will most probably be part of your course assessment portfolio. So, take care to solve as many of those tasks as possible!
- Share your work at: `v.markos@mc-class.gr`

Any Questions?

Do not forget to fill in
the questionnaire shown
right!



<https://forms.gle/dKSrmE1VRVWqxBGZA>